

R - Control IP 4 – Webhook

What is this?

The webhook reports device events **immediately to an address of your choice**. Whenever an output switches or someone presses a button, the device sends a small HTTP message to your server.

It is the easiest way to connect the device to something that speaks neither MQTT nor Modbus — for example Node-RED, ioBroker, n8n, Zapier, a home automation system or a script of your own.

How it differs from the other interfaces:

	Webhook	MQTT	Modbus / HTTP API
Direction	device → your server	both directions	your server asks the device
You need	a reachable URL	an MQTT broker	nothing else
Good for	"tell me when ..."	full integration	reading state, switching

The webhook **only reports**. You cannot switch anything through it — use the REST API, MQTT or Modbus for that.

First result in 5 minutes

Step 1: An address for testing

If you do not have a receiver yet, use a free test service such as webhook.site to try things out. The page immediately shows you your own URL, for example:

```
https://webhook.site/8f14e45f-ceeda-467a-9f3b-1a2b3c4d5e6f
```

Step 2: Enter it on the device

Open the device's web interface:

Settings → Interfaces → Webhook

1. Tick **Enable webhook**
2. Enter the **target URL** (the address from step 1)
3. Press **Save**
4. Press **Send test message**

The test message appears in your webhook.site browser tab.

Step 3: Trigger a real event

Switch an output — in the web interface under **Switch outputs** or directly on the device. Seconds later the message arrives at your receiver:

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "output",
  "action": "on",
  "index": 2,
  "name": "Yard light",
  "state": true,
  "timestamp_ms": 1789720000000,
  "uptime_s": 8641
}
```

That's it. Everything below is fine-tuning.

The settings in detail

Settings → **Interfaces** → **Webhook**

Field	Meaning
Enable webhook	Turns the function on and off
Target URL	Where messages go. Must start with http:// or https://
Secret (optional)	Sent as the header X-Webhook-Secret . Lets your server verify that a message really comes from this device
Timeout	How long the device waits for a reply (1–30 seconds, default 5)
Switch outputs	Messages when outputs are switched on/off
Switch inputs	Messages on button presses
Device start	One message when the device boots

With [https://](#) the device verifies the peer certificate against the built-in root certificates. A self-signed certificate on your server therefore **will not work** — in that case use [http://](#) on the local network, or a certificate from a well-known authority.

What a message looks like

The device sends an **HTTP POST** with [Content-Type: application/json](#).

Output switched

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "output",
  "action": "on",
  "index": 2,
  "name": "Yard light",
  "state": true,
  "timestamp_ms": 1789720000000,
  "uptime_s": 8641
}
```

Button pressed

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "input",
  "action": "click",
  "index": 1,
  "name": "Doorbell",
  "timestamp_ms": 1789720012345,
  "uptime_s": 8653
}
```

Device started

```
{
  "device": "R-Control-IP4",
  "mac": "70:B3:D5:12:34:56",
  "event": "system",
  "action": "boot",
  "timestamp_ms": 1789719991000,
  "uptime_s": 3
}
```

The fields

Field	Always present?	Meaning
device	yes	Hostname of the device (Settings → Network)
mac	yes	MAC address — unique, even if several devices share a name
event	yes	output, input, system or test
action	yes	What happened, see the table below

Field	Always present?	Meaning
<code>index</code>	for <code>output/input</code>	Number of the output or input (1–4)
<code>name</code>	for <code>output/input</code>	The name given in the settings
<code>state</code>	only for <code>output</code>	<code>true</code> = on, <code>false</code> = off
<code>timestamp_ms</code>	once the clock is set	Time in milliseconds since 1970
<code>uptime_s</code>	yes	How long the device has been running, in seconds

Possible values for `action`:

event	action	When
<code>output</code>	<code>on / off</code>	Output was switched on/off — by button, schedule, app or by hand
<code>input</code>	<code>pressed</code>	Button was pressed
<code>input</code>	<code>released</code>	Button was released
<code>input</code>	<code>long_pressed</code>	Button was held
<code>input</code>	<code>click</code>	Short click (press + release)
<code>input</code>	<code>double_click</code>	Double click
<code>system</code>	<code>boot</code>	Device has started
<code>test</code>	<code>test</code>	You pressed "Send test message"

About `timestamp_ms`: Right after power-up the device does not know the time yet. Until NTP has set the clock, the field is missing. When in doubt, rely on the time of receipt at your server.

Receiver examples

Node-RED

1. Add an **http in** node, method `POST`, URL e.g. `/rcontrol`
2. Follow it with an **http response** node, status code `200`
3. Target URL on the device: `http://<node-red-ip>:1880/rcontrol`

The **http in** node delivers the message in `msg.payload`. A function node that reacts only to output 2 being switched on:

```
if (msg.payload.event === "output" && msg.payload.index === 2 &&
msg.payload.state) {
    return msg;    // pass on
}
return null;      // drop everything else
```

Home Assistant

In `configuration.yaml`:

```
automation:
  - alias: "Doorbell pressed"
    trigger:
      platform: webhook
      webhook_id: rcontrol_event
      allowed_methods: [POST]
      local_only: true
      condition: "{{ trigger.json.event == 'input' and trigger.json.action == 'click' }}"
    action:
      - service: notify.mobile_app
        data:
          message: "Someone is at the door ({{ trigger.json.name }})"
```

Target URL on the device: `http://<homeassistant-ip>:8123/api/webhook/rcontrol_event`

For pure Home Assistant users the **MQTT interface** is usually the better choice: it creates the entities automatically and can switch as well. The webhook is useful when you want to do without a broker.

A simple PHP script

```
<?php
// receiver.php
$data = json_decode(file_get_contents('php://input'), true);

// Check the secret (optional but recommended)
$secret = $_SERVER['HTTP_X_WEBHOOK_SECRET'] ?? '';
if ($secret !== 'my-secret') {
    http_response_code(403);
    exit;
}

$line = sprintf("%s %s %s %s\n",
    date('Y-m-d H:i:s'),
    $data['event'] ?? '?',
    $data['name'] ?? '',
    $data['action'] ?? '');
file_put_contents('events.log', $line, FILE_APPEND);

http_response_code(200);
```

Trying it on your own machine

A one-liner that shows incoming requests (requires Python):

```
python -m http.server 8080
```

It only prints the request line, but that is enough to see **that** something arrives. For the full payload, webhook.site is more convenient.

What your server should answer

Reply with a status code in the **200–299** range. Anything else counts as a failure and shows up in the web interface under **Overview** → **Device status** as a notice.

Reply **quickly** — the device waits for your answer, by default up to 5 seconds. Long processing belongs after the reply, not before it.

The response body is ignored; an empty reply is fine.

Limits — please read first

No retry on failure. If your server happens to be unreachable, that one message is lost. The device does not try again.

Limited queue. Up to 8 messages wait for delivery. With very fast event sequences and a slow receiver, messages can be dropped. How often that happened is shown under **Overview** → **Device status** → **Details**.

If you need guaranteed delivery, use MQTT. A broker buffers messages and delivers them after an interruption.

The webhook is meant for the normal case: "tell my system when something happens." It is not the right tool for billing, security logs and similar.

Troubleshooting

Nothing arrives

1. Is *Enable webhook* ticked and did you press **Save**?
2. What does **Send test message** say? The status line below states the cause in plain text.
3. Look under **Overview** → **Device status** → **Details** at the *Webhook* row — it shows counters for sent/failed and the last error message.

Common messages and what they mean

Message	Cause	Remedy
ESP_ERR_HTTP_CONNECT	Server not reachable	Check IP and port; is the server on the same network?
HTTP 404	Wrong URL	Check the path in the target URL

Message	Cause	Remedy
HTTP 401 / HTTP 403	Your server rejects the request	Is the secret set? Does your server require authentication?
ESP_ERR_ESP_TLS_...	Certificate problem with https://	Self-signed certificates are not accepted — use http:// on the local network
ESP_ERR_TIMEOUT	Server replies too slowly	Increase the timeout or move processing after the reply

Messages arrive, but the time is wrong

Check under **Overview** → **Device status** whether the time is synchronized. Without NTP access the field `timestamp_ms` stays empty or contains a wrong time.

For developers: the API endpoint

The web interface's test function can also be called directly:

```
POST /api/interfaces/webhook/test
Authorization: Bearer <token>
```

Response: `{"status": "ok", "httpStatus": 200, "error": ""}`

The configuration lives in the device configuration under `config.interfaces.webhook`:

```
{
  "enabled": true,
  "url": "https://example.com/hook",
  "secret": "my-secret",
  "timeout_ms": 5000,
  "events": { "outputs": true, "inputs": true, "boot": true }
}
```

It can be read and written through `GET/POST /api/config` — see the REST API documentation.

Counters and the last error are available in `GET /api/system/status` under `interfaces.webhook`.